

PRAXIS-LEITFADEN

Git & GitHub

Versionsverwaltung, Zusammenarbeit & KI im Code-Editor

Ein Cheat-Sheet für den Einstieg — vom Installieren über das erste Repository bis zur Zusammenarbeit im Team.

Seminarkurs · CBS Stand: Juni 2026

Autor · Lisewski Noah

Inhaltsverzeichnis

TEIL 1 · GIT & GITHUB

1.1 Was ist ein Versionskontrollsystem (Git)? 4

1.2 Git & GitHub einrichten 4

2.1 Die wichtigsten Git-Befehle 6

2.2 GitHub Desktop einrichten 7

3.1 Das erste Repository erstellen 8

3.2 Code hinzufügen & committen 11

3.3 Branch veröffentlichen 13

TEIL 2 · ZUSAMMENARBEIT

4.1 Kollaborateure hinzufügen 14

4.2 Push & Pull 15

4.3 History – wer hat was geändert? 16

4.4 Branches & Merge 17

TEIL 3 · GIT ÜBER DIE KOMMANDOZEILE (CLI)

5.1 Die CLI installieren 19

5.2 Ein Repository über die CLI aufsetzen 19

TEIL 4 · WEITERE DINGE

6.1 Cloning vs. Forking 21

6.2 Eine Webseite hosten mit GitHub Pages 22

TEIL 5 · CODE-EDITOR & KI

7.1 Was ist ein Code-Editor? 24

7.2 Welche Rolle spielt KI im Code-Editor? 25

7.3 KI im Editor nutzen 26

7.4 Agentische KI: Claude Code 27

7.5 Wie professionelle Devs KI nutzen 30

ABSCHLUSS

Ende 32

Teil 1 · Git & GitHub

1.1 Was ist ein Versionskontrollsystem (Git)?

Ein Versionskontrollsystem (Version Control System, kurz VCS) verfolgt Änderungen an Dateien über die Zeit hinweg. So können mehrere Personen gemeinsam an einem Projekt arbeiten und behalten trotzdem den Überblick darüber, was sich verändert hat.

Das System zeichnet jede Aktion und jede Änderung auf – zum Beispiel jede Anpassung im Code. Dadurch können mehrere Programmierer:innen gleichzeitig am selben Code arbeiten, und jede Änderung bleibt nachvollziehbar.

GitHub ist eine cloud-basierte Plattform für Git. Sie speichert alle Änderungen in der Cloud – in sogenannten Repositories.



Git vs. GitHub

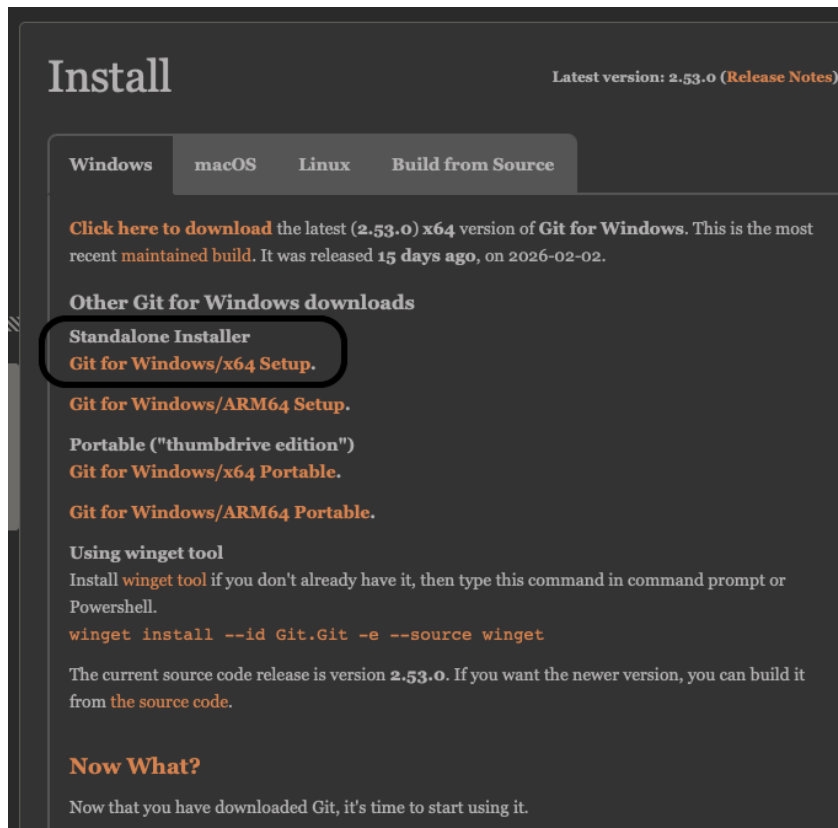
Git ist das Werkzeug auf deinem Rechner, das Änderungen aufzeichnet.

GitHub ist der Cloud-Speicher, in dem diese Änderungen für alle zugänglich liegen.

1.2 Git & GitHub einrichten

Zuerst installieren wir Git lokal auf unserem PC. Den Download findest du auf git-scm.com/downloads.

Parallel dazu legen wir ein Beispielprojekt an, damit du alles bildlich mitverfolgen kannst.



Der Download-Bereich von git-scm.com – wir wählen den markierten Standalone Installer.

Wir klicken auf den markierten Standalone Installer und laden das Setup herunter. Im Setup lassen wir alle Einstellungen auf Standard – danach ist Git installiert.

Um zu testen, ob Git wirklich auf dem PC ist, geben wir in die Kommandozeile ein:

```
git version
```

Dann sollte etwas wie das hier erscheinen:

```
git version 2.52.0
```

Geschafft – **Git** ist installiert!

Jetzt brauchen wir nur noch einen GitHub-Account. Dafür melden wir uns auf github.com an.



Tipp zur E-Mail-Adresse

Nutze am besten deine professionelle E-Mail-Adresse – Arbeitgeber werfen später gerne einen Blick auf dein GitHub-Profil.

2.1 Die wichtigsten Git-Befehle

Bevor wir praktisch loslegen, schauen wir uns die wichtigsten Git-Befehle an und wie ein Repository im Hintergrund funktioniert.

Befehl	Funktion
<code>git init</code>	Initialisiert ein Repository
<code>git add</code>	Fügt Änderungen / neue Dateien zur Stage hinzu
<code>git commit</code>	Nimmt die Änderungen als neue Version auf
<code>git clone <url></code>	Klont ein bereits existierendes Repository
<code>git reset <file></code>	Nimmt eine Datei wieder aus der Stage
<code>git status</code>	Zeigt deine aktuellen Datei-Änderungen an
<code>git commit -m 'msg'</code>	Commit zusammen mit einer Nachricht
<code>git switch -c <name></code>	Erstellt einen neuen Branch
<code>git switch <name></code>	Wechselt zwischen Branches
<code>git log main</code>	Zeigt die Historie des Haupt-Banches

Weitere Befehle findest du im offiziellen [Git Cheat Sheet](#).

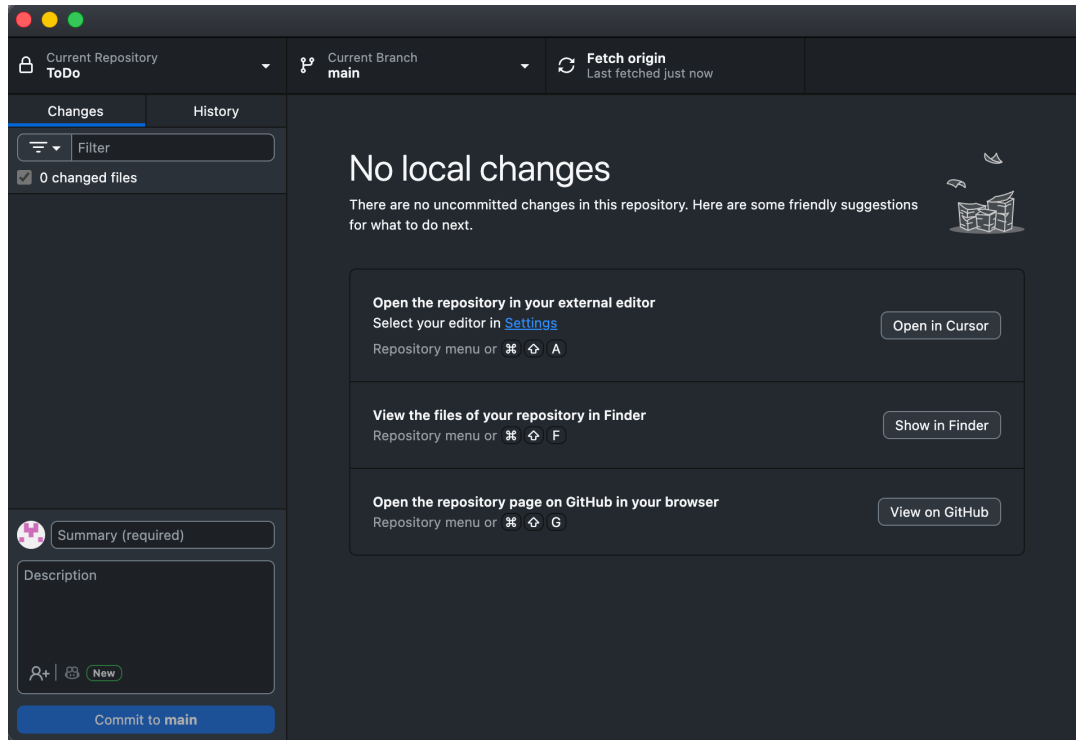


Kein Terminal nötig

In diesem Tutorial nutzen wir Git und GitHub nicht über das Terminal, sondern über ein grafisches Interface-Tool: GitHub Desktop.

2.2 GitHub Desktop einrichten

Wir holen uns das Interface-Tool [GitHub Desktop](#), durchlaufen das Setup ganz normal und melden uns mit unserem GitHub-Account an. Danach sieht das Ganze so aus:



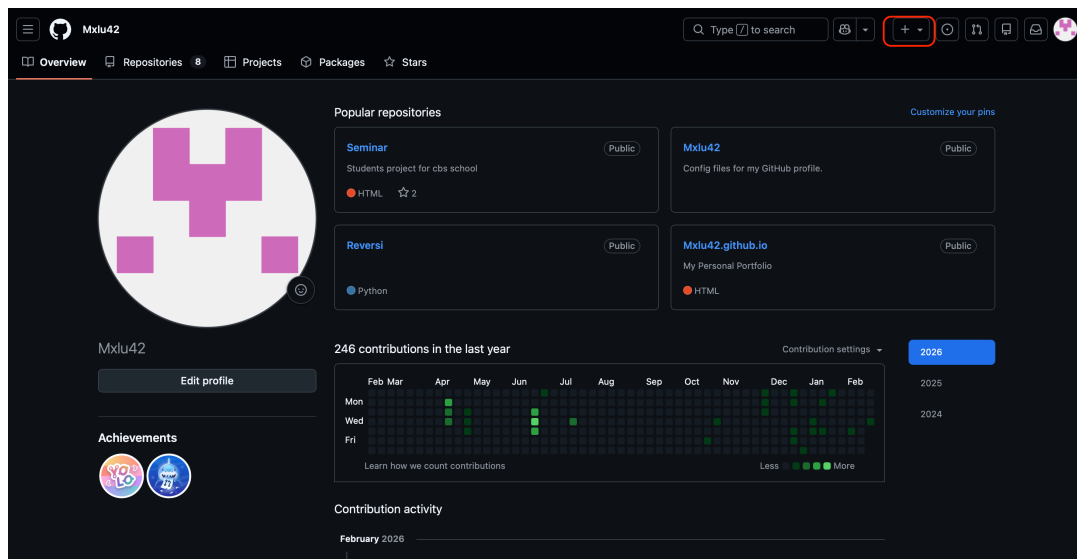
GitHub Desktop nach dem Login – noch ohne lokale Änderungen.

3.1 Das erste Repository erstellen

Jetzt erstellen wir unser erstes Repository! Ab hier lohnt es sich, genau mitzumachen, damit wir ein kleines Repository-Projekt aufbauen. Es gibt drei Möglichkeiten, ein Repository anzulegen:

- Über GitHub Desktop → Add → Create new Repository
- Über die GitHub-Website und anschließendes Klonen per GitHub Desktop
- Über das Terminal

Wir nutzen die zweite Variante, weil es so am einfachsten ist, später Kollaborateure hinzuzufügen. Dafür gehen wir auf der GitHub-Website in die Overview-Ansicht.



Die Profil-Übersicht auf GitHub.

Von dort klicken wir oben rechts auf das Plus und erstellen ein neues Repository.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * Mxlu42 / **Repository name *** Github Tutorial
Checking availability...

Great repository names are short and memorable. How about [expert-system](#)?

Description
Used as a Tutorial for my class to explain the usage of Github
62 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository **Public**

Add README Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

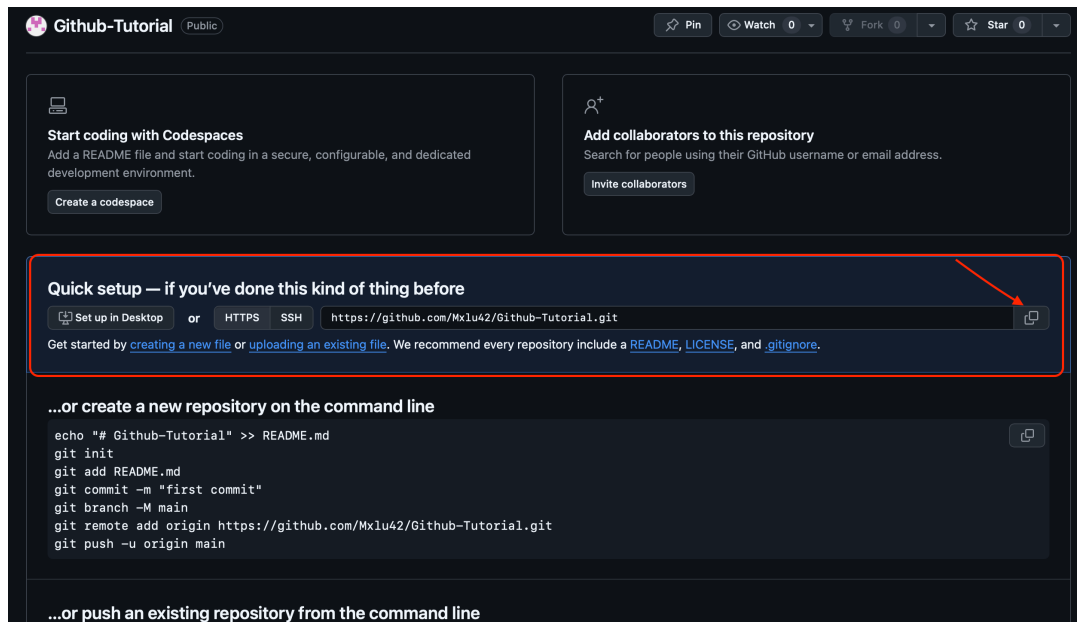
Create repository

Das Formular „Create a new repository“.

⚠ **Wichtig: Visibility auf Private**

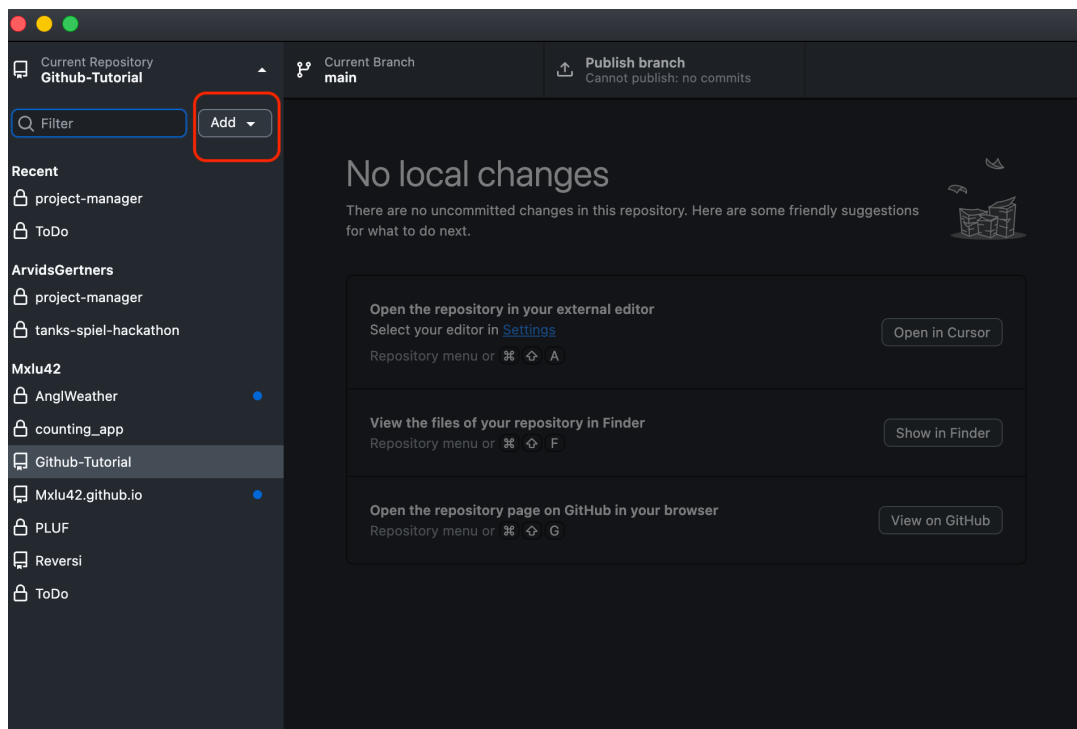
Stelle die Visibility – anders als im Bild – auf Private, damit dein Übungsversuch nicht öffentlich zugänglich ist.

Anschließend kopieren wir den HTTPS-Link des Repositories ...

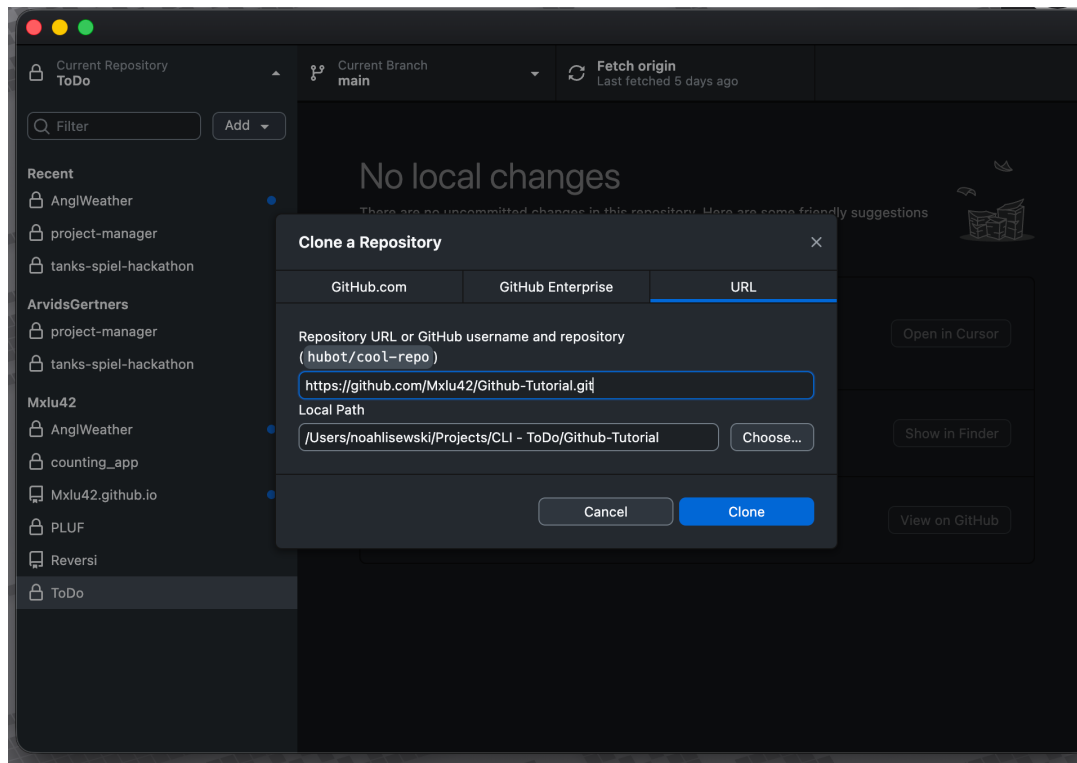


Der HTTPS-Link im „Quick setup“-Bereich.

... und fügen ihn in GitHub Desktop ein (Add → Clone repository):



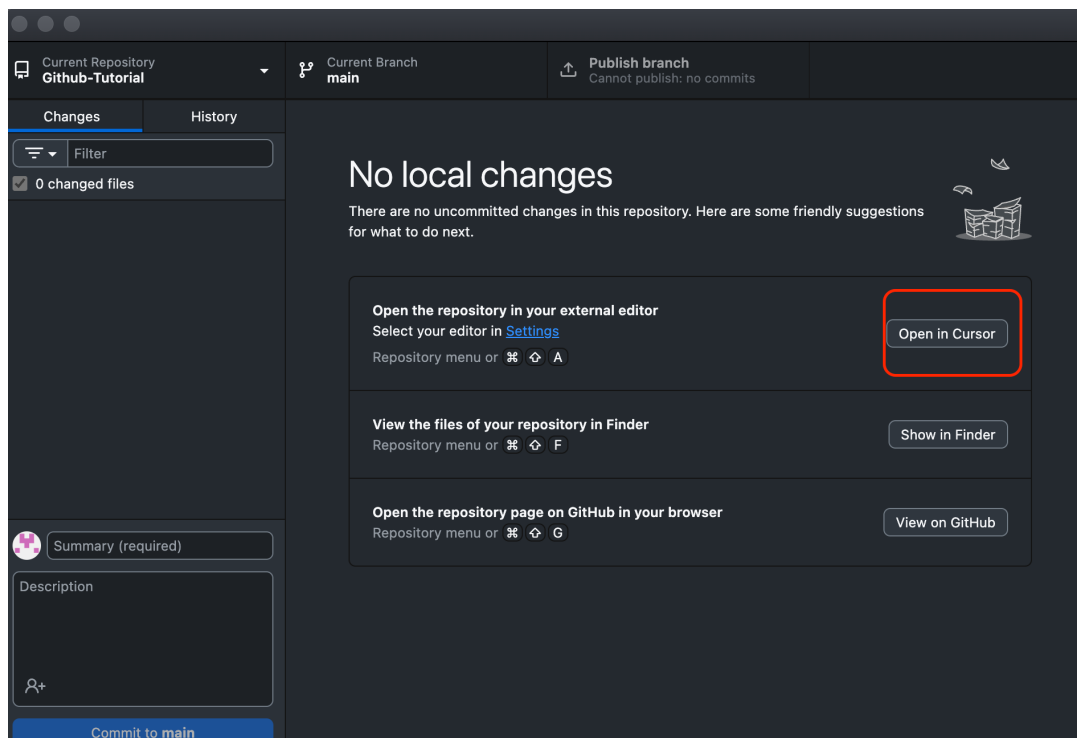
Add-Menü in GitHub Desktop.



Im Reiter „URL“ den Link einfügen und auf Clone klicken.

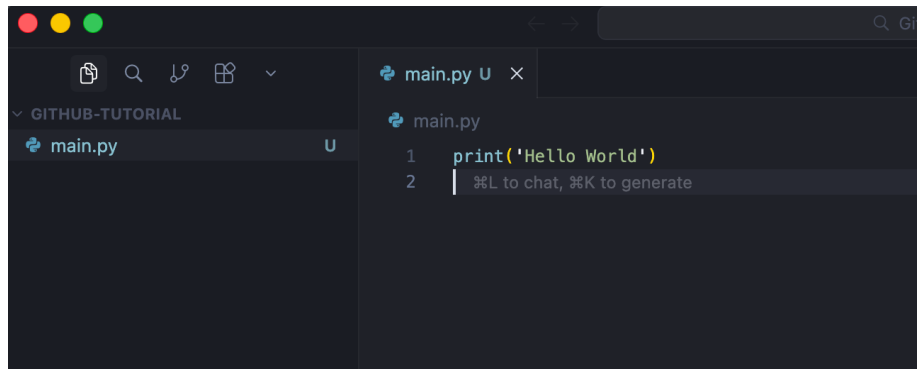
3.2 Code hinzufügen & committen

Jetzt öffnen wir über GitHub Desktop unseren Code-Editor, um im Repo Dateien hinzuzufügen und zu bearbeiten.



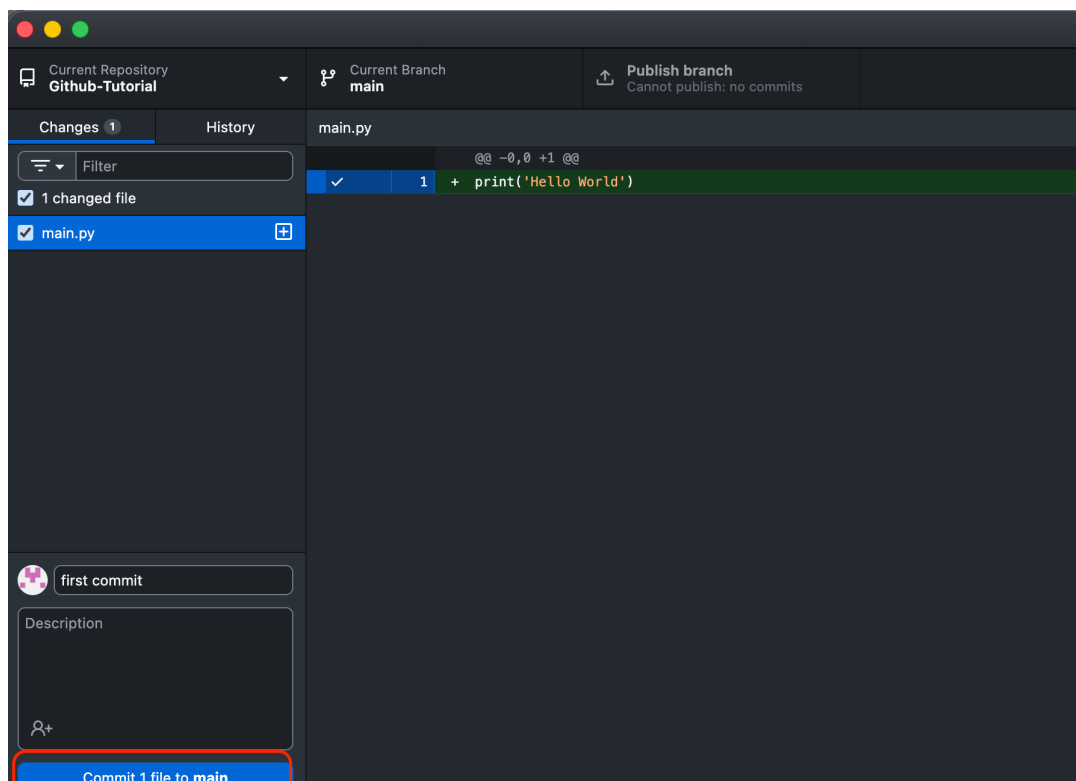
Über „Open in ...“ öffnet sich der verknüpfte Code-Editor.

Wir erstellen eine Python-Datei und schreiben unser Programm hinein:



main.py mit einem einfachen „Hello World“.

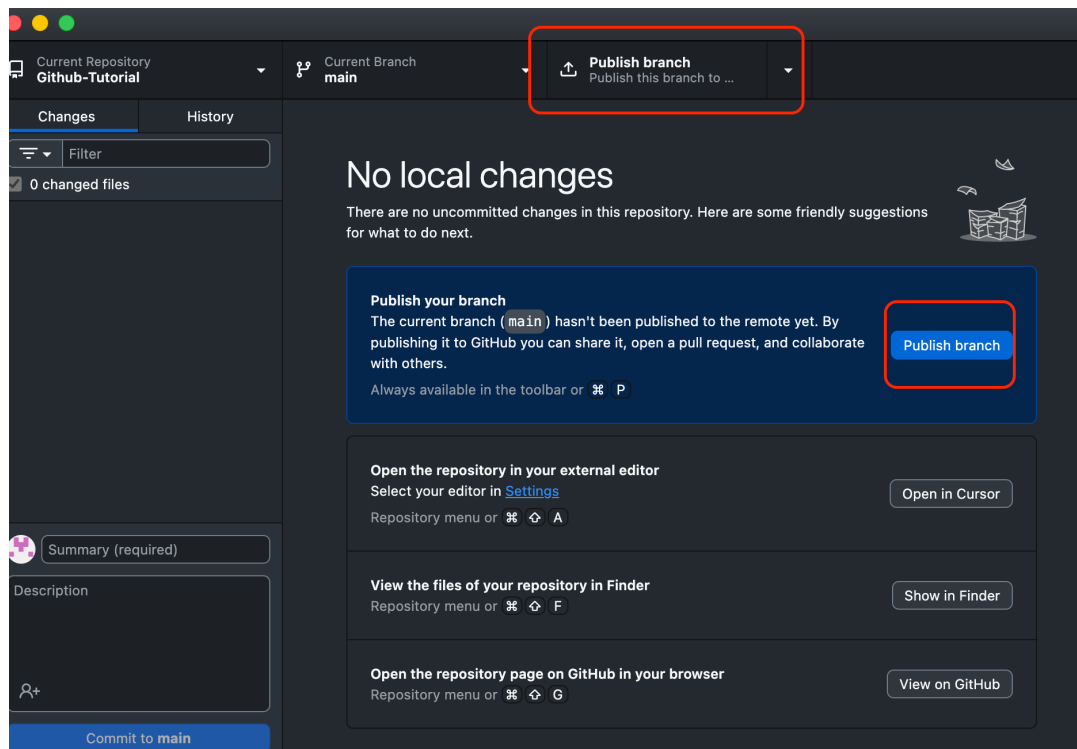
Danach committen wir die Änderung, um sie aufzunehmen. Dafür gehen wir zurück zu GitHub Desktop, schreiben unsere Commit-Nachricht und klicken auf Commit.



Die Änderung wird mit der Nachricht „first commit“ festgehalten.

3.3 Branch veröffentlichen

Anschließend veröffentlichen wir den Branch über Publish branch – erst dadurch landet er in der Cloud:



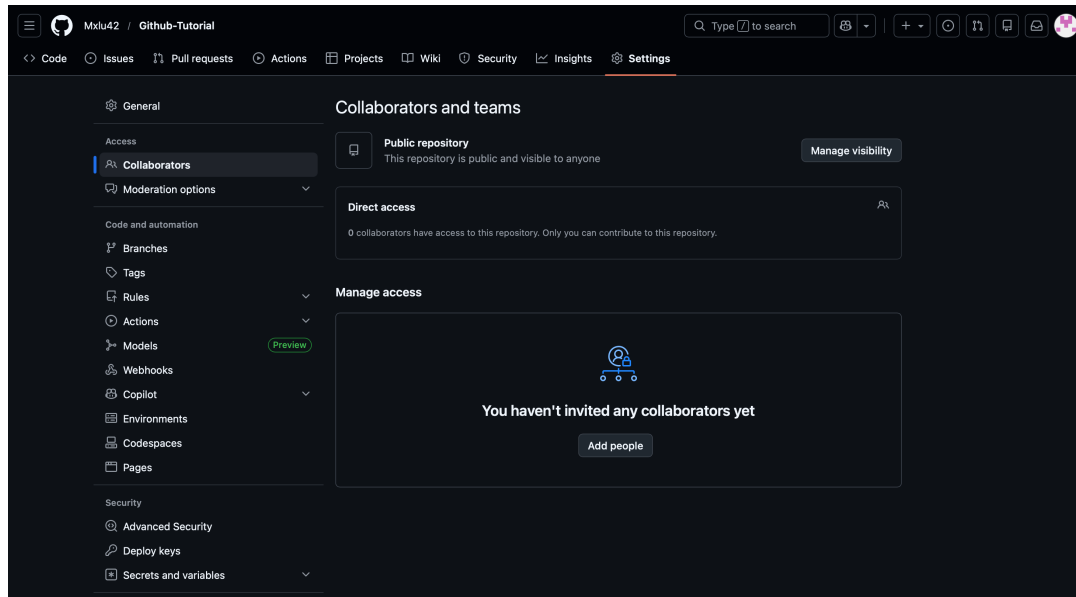
Publish branch lädt den lokalen Branch zu GitHub hoch.

Wenn wir jetzt auf **History** klicken (direkt neben Changes), sehen wir unseren ersten Commit!

Teil 2 · Zusammenarbeit

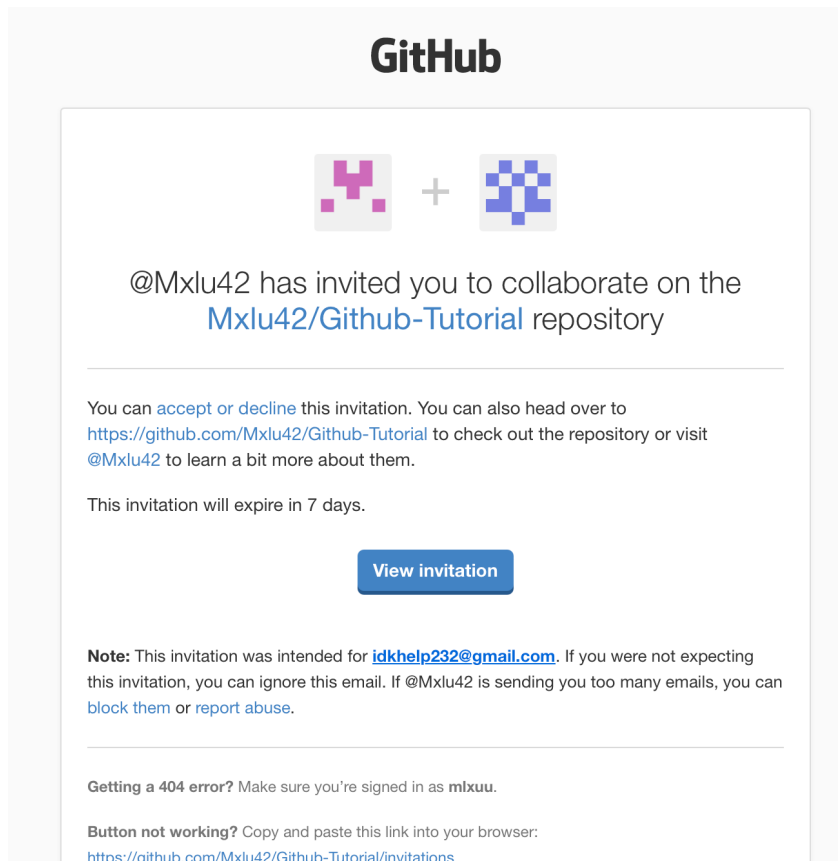
4.1 Kollaborateure hinzufügen

Auf der GitHub-Website gehen wir auf Settings → Collaborators.



Der Collaborators-Bereich in den Repository-Einstellungen.

Dann auf Add people klicken und die E-Mail oder den GitHub-Username der Partnerin / des Partners eingeben. Diese Person muss die Einladung anschließend auf der Website oder per E-Mail annehmen.



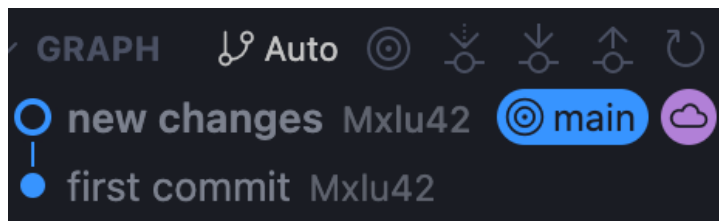
Die Einladungs-E-Mail von GitHub.

Danach klonst der Partner das Repo in seinen Code-Editor – wie das geht, ist oben bereits erklärt (siehe „Das erste Repository erstellen“). Startpunkt ist dabei das neue Repository.

4.2 Push & Pull

Bei der Zusammenarbeit nutzt man ein Push-and-Pull-System: Die eine Person pusht ihre neuen Änderungen in die Cloud, die andere pullt diese Änderungen auf ihre eigene Version herunter.

Gehen wir ein Beispiel durch: Dein Collaborator hat den Code erweitert, committet und in die Cloud gepusht.

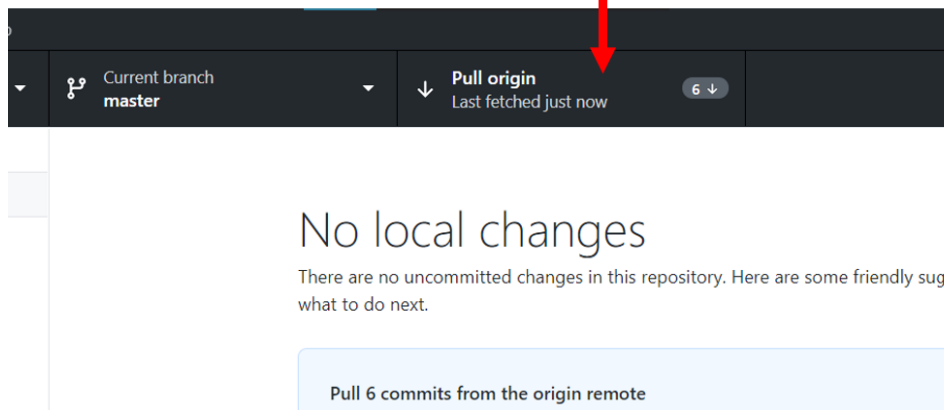


Im Graph sind die neuen Änderungen sichtbar.

Jetzt liegt die Änderung in der Cloud und muss nur noch von der anderen Seite heruntergeladen werden. Die Schritte sind:

1. Sync changes
2. Pull origin

There are 6 commits on the remote repo that need to be pulled on the local repo

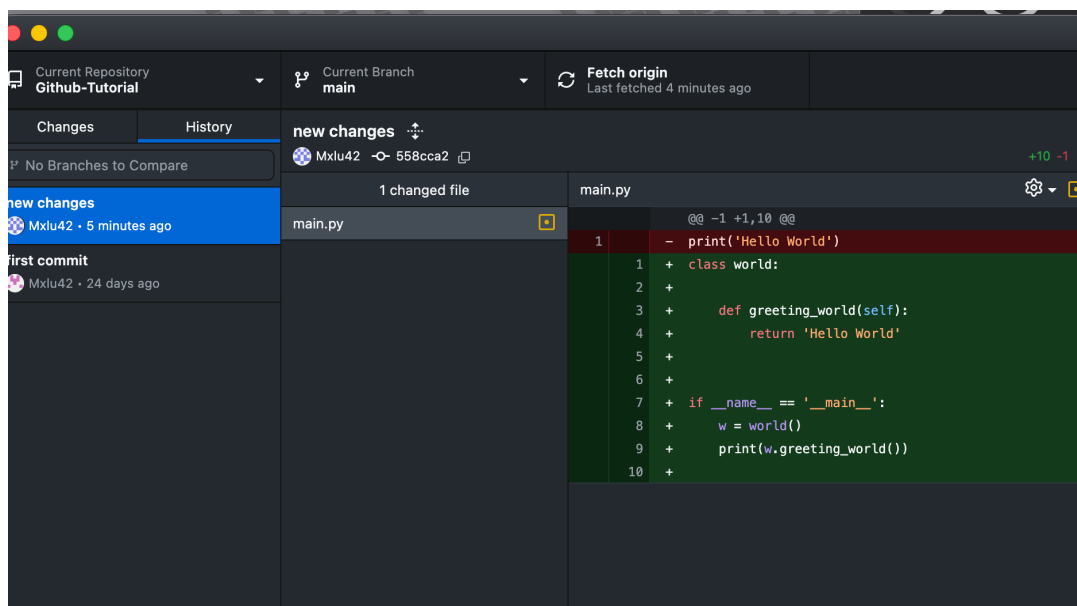


GitHub Desktop zeigt an, wie viele Commits gepullt werden müssen.

Und schon sind die Dateien lokal auf dem PC aktualisiert.

4.3 History – wer hat was geändert?

Wenn wir in die History schauen, sehen wir jede Änderung – und von wem sie gemacht wurde:

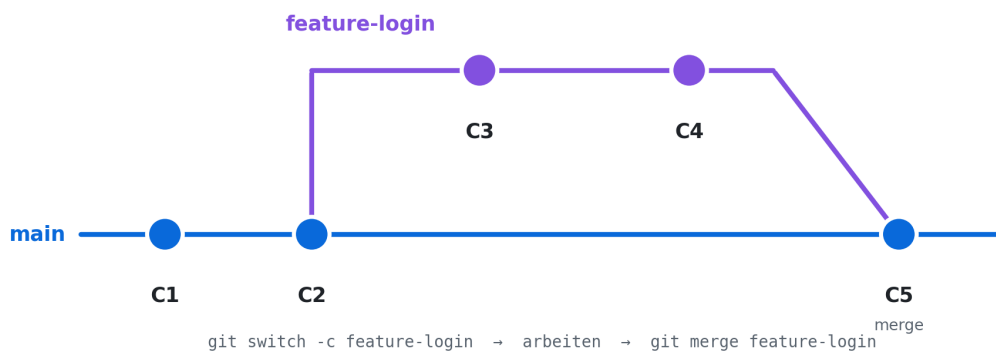


Die History zeigt den Diff und den Autor jeder Änderung.

4.4 Branches & Merge

Bisher haben wir alle Änderungen direkt auf dem main-Branch gemacht. In echten Projekten arbeitet man aber selten direkt auf main – stattdessen erstellt man für jede neue Funktion einen eigenen Branch.

Ein Branch ist wie eine parallele Kopie deines Codes, in der du gefahrlos experimentieren kannst, ohne den funktionierenden Stand auf main zu zerstören. Wenn du fertig bist, führst du den Branch per Merge wieder mit main zusammen.



Ein feature-Branch zweigt von main ab und wird später wieder zurückgemergt.

Über die Kommandozeile sieht das so aus:

```

git switch -c feature-login  # neuen Branch erstellen + hineinwechseln
# ... Code ändern, git add, git commit ...

git switch main              # zurück auf main
git merge feature-login      # Branch in main zusammenführen

```



Merge in GitHub Desktop

Lieber grafisch? In GitHub Desktop geht das über Branch → Merge into current branch. Vorher den Zielbranch (main) auswählen.

Merge-Konflikte

Ein Merge-Konflikt entsteht, wenn zwei Branches dieselbe Zeile unterschiedlich verändert haben. Git kann dann nicht automatisch entscheiden, welche Version gewinnt, und markiert die Stelle so:

```
<<<<<<< HEAD
print('Hallo Welt')
=====
print('Hello World')
>>>>>> feature-login
```

Du löschst die Markierungen, behältst die richtige Version (oder kombinierst beide) und machst einen neuen Commit. GitHub Desktop bietet dir dafür sogar einen visuellen Editor an.

(Visuellen Editor Screenshot)

Teil 3 · Git über die Kommandozeile (CLI)

Bisher haben wir alles bequem über GitHub Desktop gemacht. Profis arbeiten aber oft direkt im Terminal – das ist schneller und funktioniert auf jedem Rechner, sogar auf Servern ganz ohne grafische Oberfläche. Hier zeigen wir denselben Ablauf wie in Teil 1, diesmal komplett über die CLI.

5.1 Die CLI installieren

Git bringt seine Kommandozeilen-Befehle schon mit – die haben wir in Kapitel 1.2 installiert. Optional gibt es zusätzlich die GitHub CLI: das offizielle Tool `gh`, mit dem du direkt aus dem Terminal mit GitHub sprichst (Repos erstellen, Pull Requests öffnen usw.).

```
# macOS (mit Homebrew)
brew install gh

# Windows (mit winget)
winget install --id GitHub.cli

# danach einmalig anmelden:
gh auth login
```

`gh auth login` führt dich Schritt für Schritt durch die Anmeldung. Ob alles geklappt hat, prüfst du mit:

```
gh --version
```



Brauche ich `gh` wirklich?

Nein. Reines `git` reicht für alles Folgende völlig aus. `gh` macht GitHub-Aktionen nur bequemer – du kannst es jederzeit später nachinstallieren.

5.2 Ein Repository komplett über die CLI aufsetzen

Jetzt bauen wir dasselbe Projekt wie in Teil 1 – nur mit dem Terminal. Wir legen einen Ordner an, initialisieren Git, fügen eine Datei hinzu und pushen alles zu GitHub.

```
# 1. Projektordner erstellen und hineinwechseln
mkdir github-tutorial
cd github-tutorial

# 2. Git-Repository initialisieren
git init

# 3. Eine Datei anlegen
echo "# Github-Tutorial" > README.md

# 4. Datei zur Stage hinzufügen
git add README.md

# 5. Ersten Commit erstellen
git commit -m "first commit"

# 6. Haupt-Branch auf 'main' setzen
git branch -M main
```

Jetzt verbinden wir das lokale Repo mit GitHub. Entweder du legst vorher auf github.com ein leeres Repository an und kopierst den Link – oder du erledigst alles direkt mit gh:

```
# Variante A: Repo wurde vorher auf github.com angelegt
git remote add origin https://github.com/DEIN-NAME/github-tutorial.git
git push -u origin main

# Variante B: Repo direkt aus dem Terminal erstellen (mit gh)
gh repo create github-tutorial --private --source=. --push
```



Das -u merkt sich die Verbindung

Durch `git push -u origin main` merkt sich Git die Verbindung zwischen lokalem und entferntem Branch. Danach reicht in Zukunft ein einfaches `git push` bzw. `git pull`.

Übrigens: Genau diese Befehlsliste hast du schon einmal gesehen – GitHub zeigt sie dir an, wenn du ein neues, leeres Repo erstellst (siehe Screenshot in Kapitel 3.1, Bereich „...or create a new repository on the command line“).

Teil 4 · Weitere Dinge

6.1 Cloning vs. Forking

Beide Begriffe bedeuten „sich eine Kopie eines Repositories holen“ – aber es gibt einen wichtigen Unterschied:

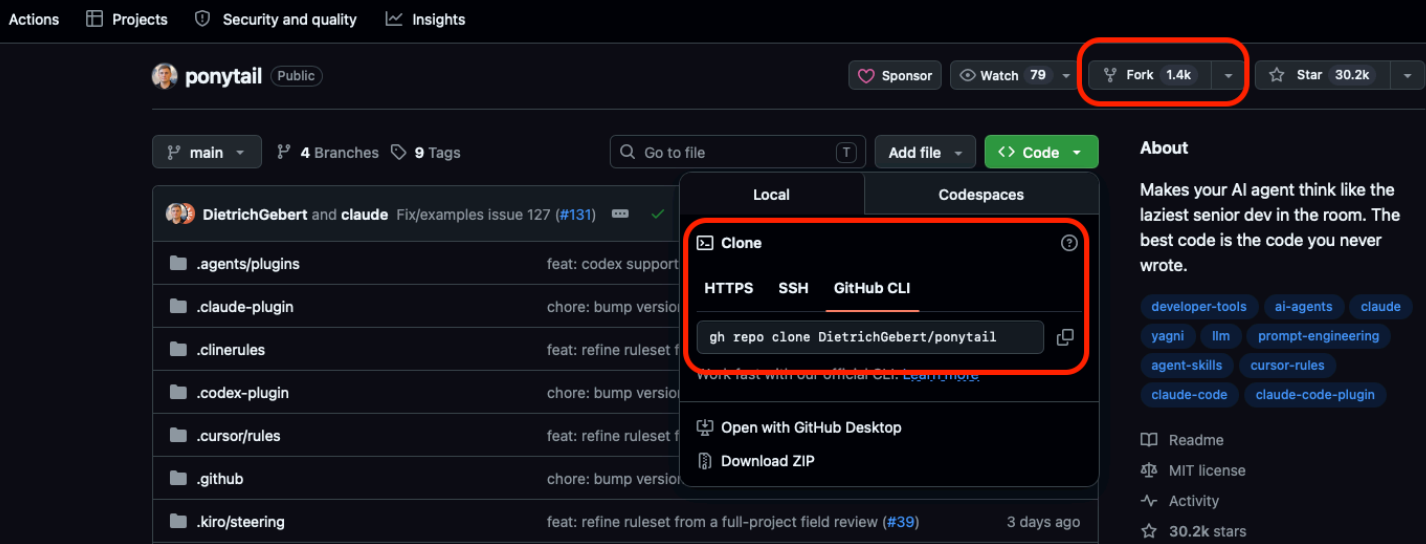
	Clone	Fork
Was passiert?	Kopie auf deinen PC (lokal)	Kopie in deinen GitHub-Account (Cloud)
Wofür?	An einem Repo arbeiten, auf das du Zugriff hast	Ein fremdes Repo verändern, ohne Schreibrechte
Verbindung	bleibt dasselbe Repo	eigene Kopie, per Pull Request verbunden
Aktion	<code>git clone <url></code>	„Fork“-Button auf GitHub

Faustregel

Bist du Collaborator? → **Clone**. Willst du zu einem fremden (z. B. Open-Source-)Projekt beitragen? → **Fork**.

Der typische Open-Source-Ablauf sieht so aus:

```
# 1. Auf GitHub den Fork-Button klicken -> eigene Kopie entsteht
# 2. Deine Kopie lokal klonen
git clone https://github.com/DEIN-NAME/fremdes-projekt.git
# 3. Änderungen machen, committen, pushen
git push
# 4. Auf GitHub: "Compare & pull request" -> Pull Request öffnen
```

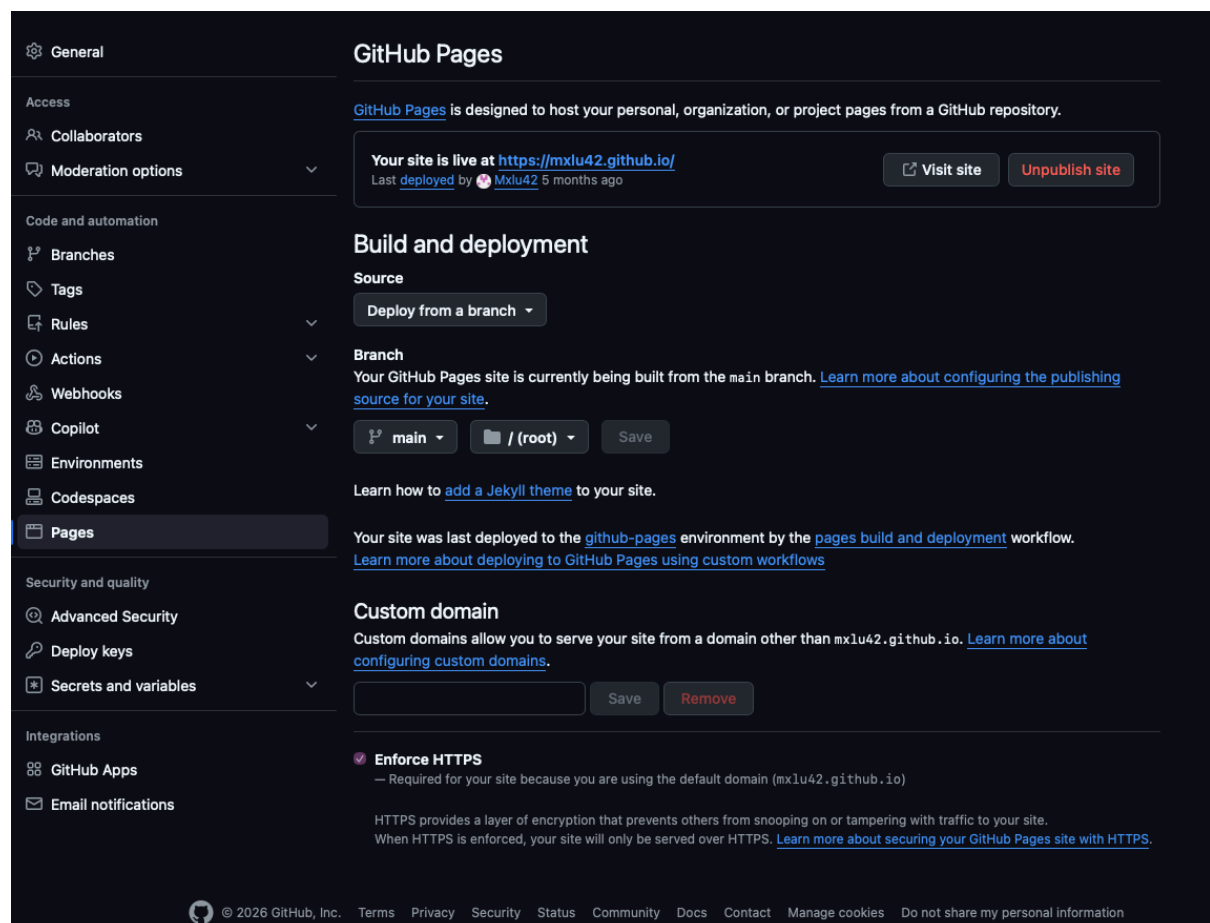


6.2 Eine Webseite hosten mit GitHub Pages

GitHub kann deine HTML-Dateien kostenlos als echte Webseite im Internet veröffentlichen – das nennt sich GitHub Pages. Perfekt für ein Portfolio oder eine Projektseite.

1. Lege ein Repository an (oder nutze ein bestehendes) mit einer index.html.
2. Gehe im Repo auf Settings → Pages.
3. Wähle unter „Branch“ deinen main-Branch und den Ordner / (root), dann Save.
4. Nach ein bis zwei Minuten ist deine Seite erreichbar unter:

`https://DEIN-NAME.github.io/REPO-NAME/`



Eine minimale index.html, mit der du sofort starten kannst:

```
<!DOCTYPE html>

<html>

  <head><title>Meine Seite</title></head>

  <body>

    <h1>Hallo Welt!</h1>

    <p>Gehostet mit GitHub Pages.</p>

  </body>

</html>
```



Sonderfall: Portfolio-URL

Heißt dein Repo genau `DEIN-NAME.github.io`, ist die Seite direkt unter `https://DEIN-NAME.github.io/` erreichbar – ohne Repo-Namen am Ende. Genau so funktioniert das Portfolio aus dem Profil-Screenshot in Kapitel 3.1.

⚠ Visibility beachten

GitHub Pages funktioniert nur mit öffentlichen Repos (oder mit GitHub Pro). Ein reines Übungs-Repo, das auf Private steht, musst du dafür also auf Public umstellen.

Teil 5 · Code-Editor & KI

Bis hierher ging es um Git und GitHub. Der zweite große Block dreht sich darum, wo wir den Code überhaupt schreiben – im Code-Editor – und wie KI dort heute zum festen Werkzeug geworden ist.

7.1 Was ist ein Code-Editor?

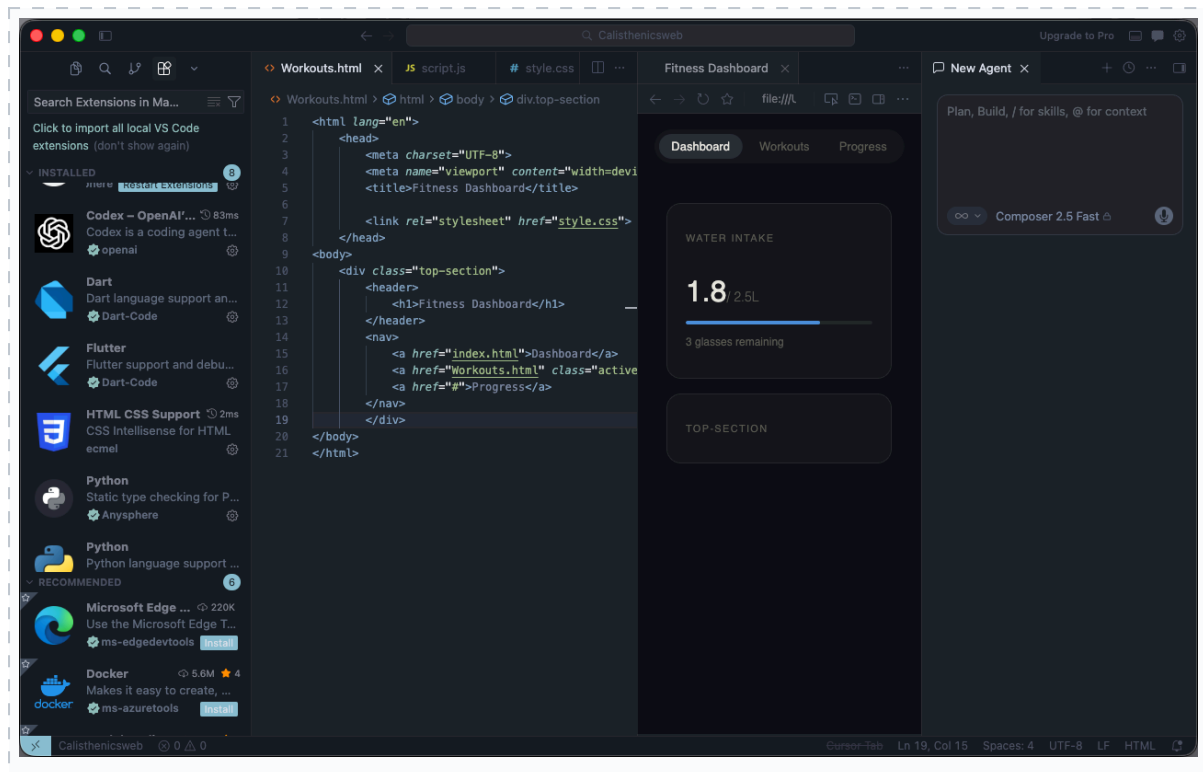
Ein Code-Editor ist ein Programm zum Schreiben und Bearbeiten von Quellcode. Im Gegensatz zu einem normalen Texteditor bringt er Funktionen mit, die das Programmieren enorm erleichtern:

- Syntax-Highlighting – Code wird farbig hervorgehoben, sodass Struktur und Fehler sofort sichtbar sind
- Autovervollständigung – Vorschläge beim Tippen von Befehlen und Variablennamen
- Integriertes Terminal – Befehle (z. B. git) direkt im Editor ausführen
- Git-Integration – Änderungen, Commits und Branches sehen, ohne den Editor zu verlassen
- Erweiterungen (Extensions) – zusätzliche Sprachen, Werkzeuge und KI nachrüsten

Die bekanntesten Vertreter sind **VS Code** (kostenlos, mit Abstand am weitesten verbreitet) und **Cursor** – ein auf VS Code aufbauender Editor, bei dem KI von Anfang an fest eingebaut ist. Cursor haben wir in diesem Tutorial übrigens schon benutzt (siehe „Open in Cursor“ in Kapitel 3.2).

Editor vs. IDE

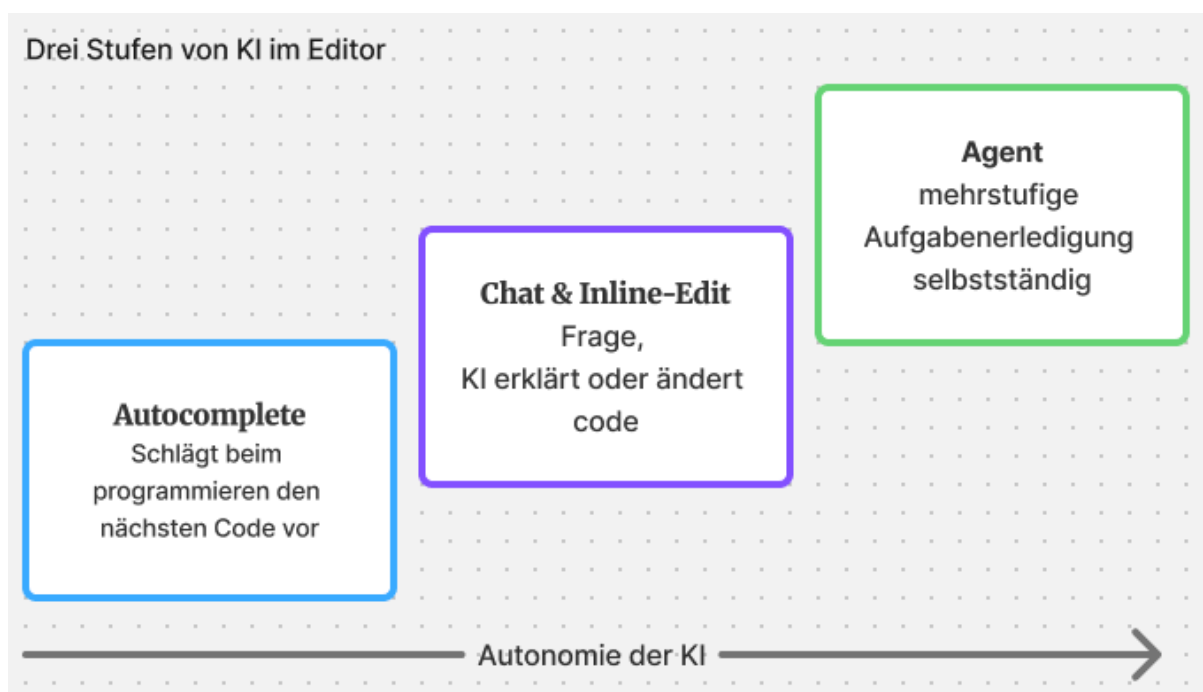
Eine IDE (Integrated Development Environment, z. B. die JetBrains-Tools) bringt zusätzlich Debugger, Build-Werkzeuge usw. von Haus aus mit. Ein moderner Editor wie VS Code wird durch Extensions praktisch zur IDE – die Grenze ist heute fließend.



7.2 Welche Rolle spielt KI im Code-Editor?

Früher war die „Intelligenz“ eines Editors reine Autovervollständigung: Er kannte deine Variablennamen. Heute schreibt, erklärt und verändert KI ganzen Code. Dabei lassen sich grob drei Stufen unterscheiden, die sich in der Autonomie unterscheiden:

Drei Stufen von KI im Editor – von Vorschlägen bis zum selbstständigen Agenten.



Der Nutzen liegt auf der Hand: KI schreibt Routine-Code (Boilerplate), findet Fehler, erklärt fremden Code und beschleunigt vieles. Gleichzeitig gilt: KI macht Fehler und kann plausibel klingenden, aber falschen Code liefern. Du bleibst verantwortlich und solltest jede Änderung prüfen – mehr dazu in 7.5.

7.3 KI im Editor nutzen

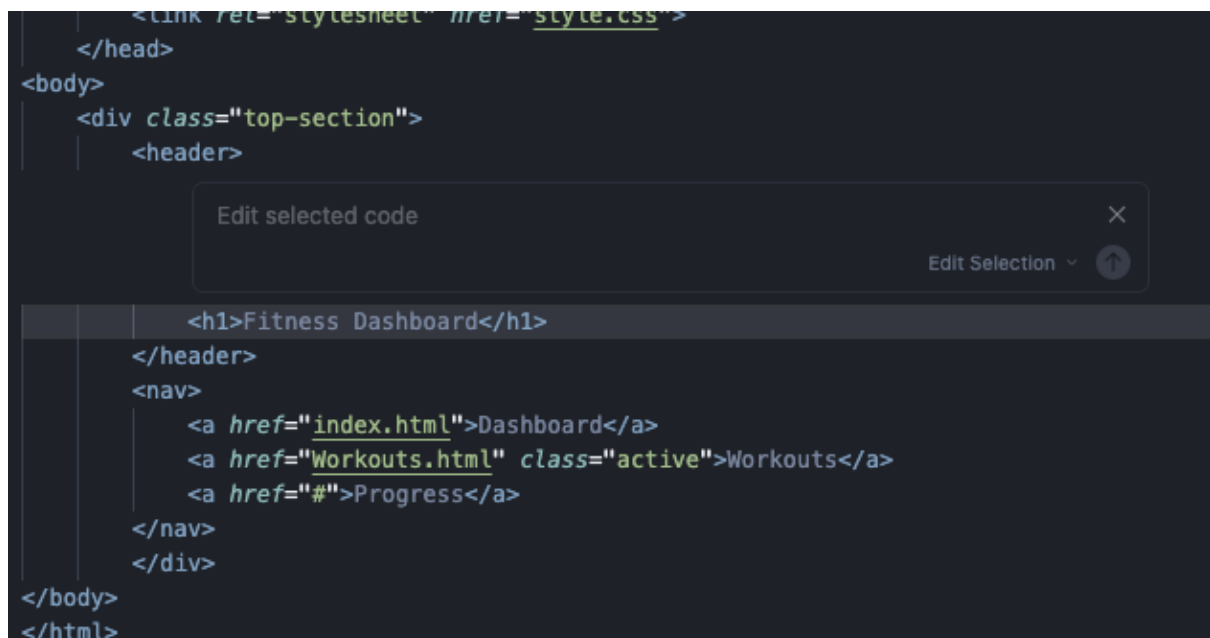
Schauen wir uns die zwei alltäglichsten Stufen konkret an. Die Tastenkürzel beziehen sich auf Cursor; in VS Code funktioniert es mit einer KI-Extension (z. B. GitHub Copilot oder der Claude-Erweiterung) sehr ähnlich.

Autocomplete (Tab)

Du tippst – die KI schlägt grau den nächsten Code vor. Passt der Vorschlag, übernimmst du ihn mit der Tab-Taste. Das spart besonders bei wiederkehrenden Mustern viel Tipparbeit.

Inline-Edit (⌘K / Strg+K)

Markiere eine Stelle im Code, drücke ⌘K (macOS) bzw. Strg+K (Windows) und beschreibe in Worten, was passieren soll – z. B. „mach daraus eine Funktion“ oder „füge Fehlerbehandlung hinzu“. Die KI schreibt den Code direkt um und zeigt dir den Unterschied (Diff) zum Annehmen oder Verwerfen.



```
<link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="top-section">
    <header>
      <h1>Fitness Dashboard</h1>
    </header>
    <nav>
      <a href="index.html">Dashboard</a>
      <a href="Workouts.html" class="active">Workouts</a>
      <a href="#">Progress</a>
    </nav>
  </div>
</body>
</html>
```

Chat (⌘L / Strg+L)

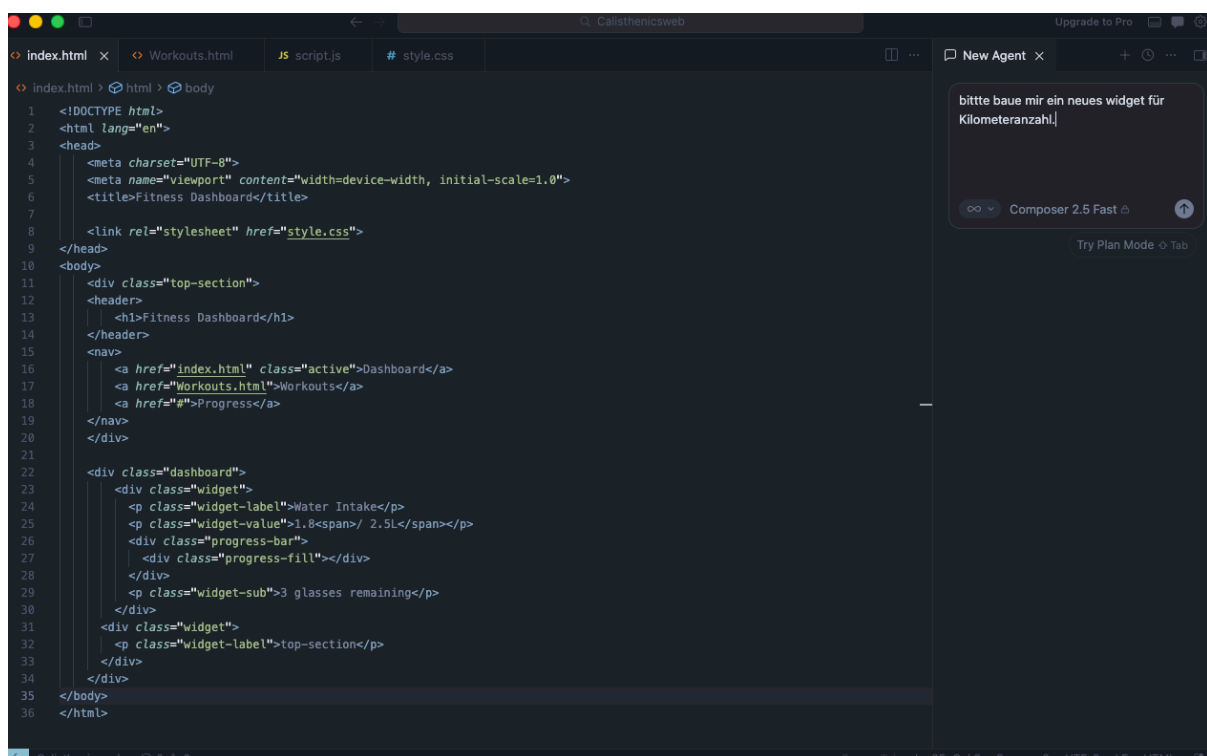
Über ein Seitenpanel führst du ein Gespräch über deinen Code: „Erklär mir diese Funktion“, „Warum stürzt das Programm ab?“, „Schreib mir einen Test dafür“. Mit @ kannst du dabei gezielt auf einzelne Dateien verweisen, damit die KI den richtigen Kontext hat.

Beispiele für gute Chat-Prompts:

Erklär mir, was main.py Zeile für Zeile macht.

Finde den Bug in @main.py und schlage einen Fix vor.

Schreib einen Unit-Test für die Funktion greeting_world().



7.4 Agentische KI: Claude Code

Stufe 3 ist der Sprung vom Assistenten zum Agenten. Claude Code ist ein agentisches Kommandozeilen-Tool von Anthropic: Es liest dein Repository, schreibt und ändert Dateien, führt Befehle aus und kann sogar selbst committen – es erledigt also ganze Aufgaben, statt nur einzelne Vorschläge zu machen.

⚠ Voraussetzung

Claude Code braucht ein bezahltes Konto (Claude Pro, Max, Team, Enterprise) oder einen Anthropic-Console-Account. Der kostenlose Claude-Plan reicht nicht aus.

Installieren

Anthropic empfiehlt heute den nativen Installer – dafür ist kein Node.js nötig:

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

# Alternativen:
brew install --cask claude-code          # macOS (Homebrew)
winget install Anthropic.ClaudeCode      # Windows (WinGet)
npm install -g @anthropic-ai/claude-code # benötigt Node.js 18+
```

Danach prüfst du die Installation mit:

```
claude --version    # zeigt die Version
claude doctor       # prüft die Installation gründlicher
```



Kein Terminal-Fan?

Es gibt Claude Code auch als Desktop-App und im Browser („Claude Code on the web“) – damit kannst du die agentische KI ganz ohne Kommandozeile nutzen.

Die erste Session

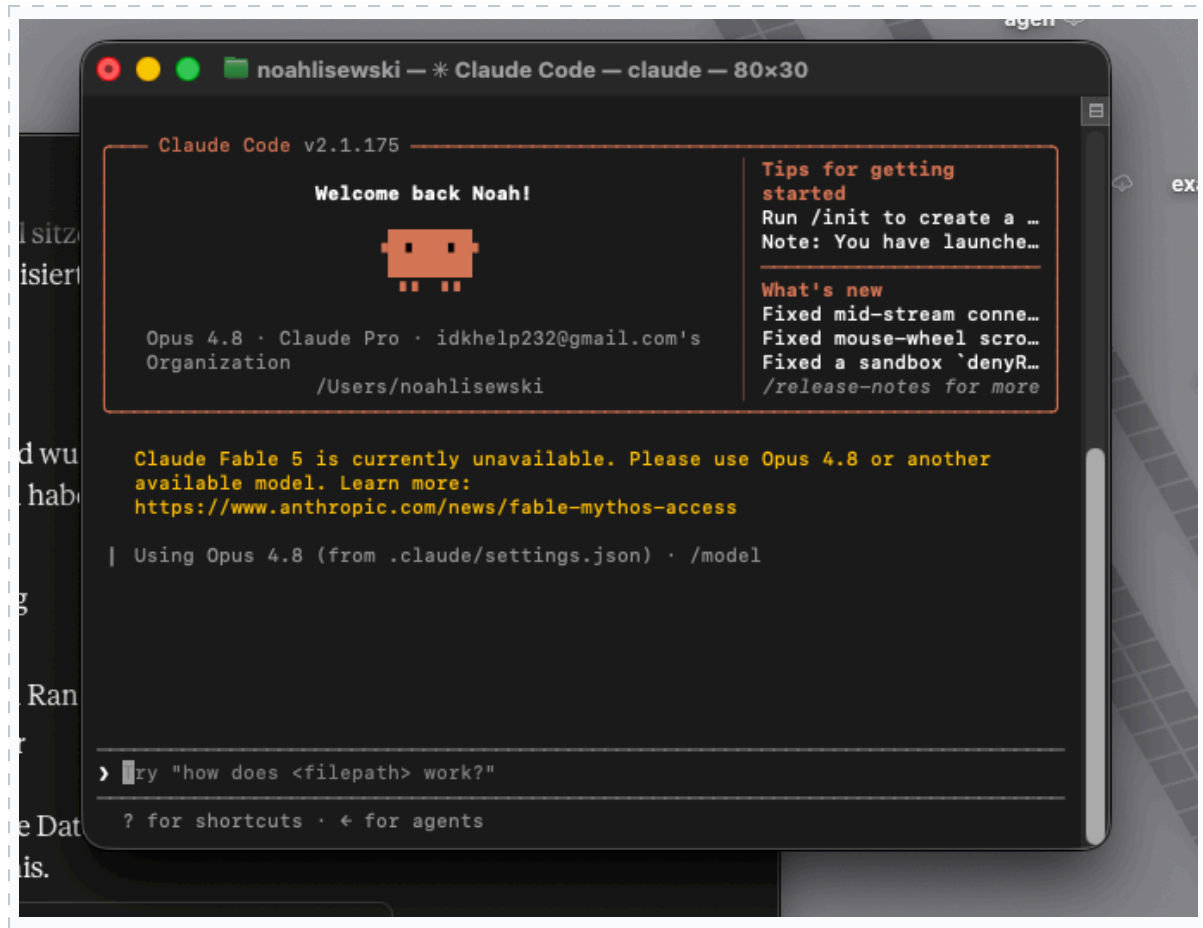
Du wechselst in deinen Projektordner und startest Claude Code. Beim ersten Mal meldest du dich über den Browser an:

```
cd github-tutorial  # in dein Projekt wechseln
claude              # Claude Code starten
```

Jetzt sprichst du einfach in natürlicher Sprache mit ihm, zum Beispiel:

```
> Erweitere main.py um eine Funktion, die den Nutzer begrüßt,  
und schreibe einen Test dafür.
```

Claude Code zeigt dir, welche Dateien es ändern will, macht die Änderungen und fragt bei wichtigen Schritten (z. B. Befehle ausführen) nach Bestätigung.



Wichtige Slash-Befehle

Tippst du im Chat ein /, erscheint eine Liste eingebauter Befehle. Die wichtigsten für den Anfang:

Befehl	Funktion
<code>/init</code>	Erzeugt eine CLAUDE.md – das „Projektgedächtnis“
<code>/help</code>	Zeigt die verfügbaren Befehle
<code>/model</code>	Modell wählen (z. B. Opus, Sonnet, Haiku)
<code>/plan</code>	Erst einen Plan erstellen lassen, dann umsetzen
<code>/review</code>	Den aktuellen Code / Diff prüfen lassen
<code>/clear</code>	Den Gesprächskontext zurücksetzen

Befehl	Funktion
<code>/resume</code>	Eine frühere Sitzung fortsetzen
<code>/exit</code>	Claude Code beenden



CLAUDE.md – das Projektgedächtnis

Mit `/init` legt Claude Code eine Datei `CLAUDE.md` an. Darin stehen deine Projekt-Infos und Regeln (z. B. „nutze Deutsch in Kommentaren“). Claude liest sie zu Beginn jeder Sitzung – so musst du den Kontext nicht jedes Mal neu erklären.

Und der Bogen zu Teil 1–4: Claude Code beherrscht Git selbst. Es kann `git add`, `git commit` und `git push` für dich ausführen – mit sinnvollen Commit-Nachrichten. Genau deshalb ist das Git-Wissen aus den ersten Kapiteln so wichtig: Du verstehst, was die KI im Hintergrund tut.

7.5 Wie professionelle Devs KI nutzen

In der Praxis ist KI längst Teil des Alltags vieler Entwickler:innen. Typische Einsätze:

- Tests schreiben und ausführen lassen
- Bugs suchen und beheben („Warum schlägt dieser Test fehl?“)
- Refactoring – unübersichtlichen Code aufräumen, ohne das Verhalten zu ändern
- Code-Reviews – einen Pull Request prüfen und Verbesserungen vorschlagen
- Ein fremdes Repo verstehen („Erklär mir, wie dieses Projekt aufgebaut ist“)
- Routinearbeit – Boilerplate, Commit-Nachrichten und Doku generieren

Der typische Arbeitsablauf bleibt dabei immer derselbe – ein Kreislauf aus Mensch und KI:

1. Aufgabe klar und klein formulieren
2. Die KI plant und arbeitet (bei Claude Code: `/plan`, dann umsetzen)
3. Den Diff prüfen – verstehst du jede Änderung?
4. Im Git-Branch committen (so kannst du jederzeit zurück)

**Best Practices**

Stelle kleine, klar umrissene Aufgaben statt „bau mir die ganze App“.

Arbeite in einem eigenen Branch – dann kannst du Fehlentscheidungen einfach verwerfen.

Hinterlege Konventionen in der CLAUDE.md, damit die KI deinen Stil trifft.

Lies und verstehe jede Änderung, bevor du sie übernimmst.

⚠ Vertraue nicht blind

KI kann überzeugend klingende, aber falsche Lösungen liefern („Halluzinationen“). Gib keine Passwörter, API-Keys oder sensiblen Daten weiter, und denke daran: Verantwortlich für den Code bleibst du, nicht die

Und genau hier schließt sich der Kreis zu Teil 1–4: Git und GitHub sind die Grundlage, die dir die Kontrolle gibt. Branches, Commits und die History sorgen dafür, dass du jederzeit nachvollziehen und zurücknehmen kannst, was die KI geschrieben hat – egal wie viel sie für dich übernimmt.

Ende

Damit ist das komplette Inhaltsverzeichnis abgedeckt – von der ersten Git-Installation bis zur agentischen KI im Editor. Du kennst jetzt:

- Git & GitHub: Versionsverwaltung, das erste Repository, Commits und History
- Zusammenarbeit: Kollaborateure, Push & Pull, Branches und Merge
- Die Kommandozeile: Git und ein Repo komplett über das Terminal
- Weitere Dinge: Cloning vs. Forking und Webseiten-Hosting mit GitHub Pages
- Code-Editor & KI: von Autocomplete über Chat bis zu Claude Code

Der beste nächste Schritt ist jetzt: ein eigenes kleines Projekt anlegen, ein paar Commits machen und die KI im Editor ausprobieren. Viel Erfolg!



Offizielle Quellen zum Weiterlesen

Git Cheat Sheet: education.github.com · Claude Code: code.claude.com/docs